src\App.java

```java
import java.util.Scanner;
import java.util.ArrayList;

import model.Account;
import model.SavingAccount;
import model.InvestmentAccount;
import model.Bank;
import model.Client;
import model.Employed;
import model.Investment;
import model.Investor;

public class App {
    public static void main(String[] args) throws Exception {
        Scanner sn = new Scanner(System.in);
        // Inicialite the bank with empty lists for clients, employees, and investments
        Bank myBank = new Bank("My Bank", "Calle 34 #5-45", new ArrayList<>(), new ArrayList<>(), new ArrayList<>());
        boolean exit = false;
        int option;

        while (!exit) {
            System.out.println("\n==== BIENVENIDO A " + myBank.getName().toUpperCase() + " ====");
            System.out.println("1. Registrar Empleado");
            System.out.println("2. Registrar Cliente");
            System.out.println("3. Registrar Casa Inversionista y Oferta");
            System.out.println("4. Realizar Depósito");
            System.out.println("5. Realizar Retiro (Requiere ID Física)");
            System.out.println("6. Ver Catálogo de Inversiones");
            System.out.println("7. Consultar Días de Vacaciones");
            System.out.println("8. Aplicar Intereses Mensuales (Solo Ahorro)");
            System.out.println("9. Salir");
            System.out.print("Seleccione una opción: ");

            option = sn.nextInt();
            sn.nextLine(); // Clear buffer

            switch (option) {
                case 1:
                    System.out.println("\n--- Registro de Empleado ---");
                    System.out.print("Nombre: ");
                    String empName = sn.nextLine();
                    System.out.print("Apellido: ");
                    String empLastName = sn.nextLine();
                    System.out.print("ID: ");
                    String empId = sn.nextLine();
                    System.out.print("ID Usuario: ");
```

```java
                    String empUser = sn.nextLine();
                    System.out.print("Número de cuenta nómina: ");
                    String empAcc = sn.nextLine();
                    System.out.print("Puesto (Cajero/Supervisor/Recepcionista): ");
                    String pos = sn.nextLine();
                    System.out.print("Años trabajando: ");
                    int yrs = sn.nextInt();

                    Employed newEmp = new Employed(empName, empLastName, empId, empUser, empAcc, pos, yrs);
                    myBank.addEmployed(newEmp);
                    System.out.println("Empleado registrado y cuenta de nómina activada.");
                    break;

                case 2:
                    System.out.println("\n--- Registro de Cliente ---");
                    System.out.print("Nombre: ");
                    String cliName = sn.nextLine();
                    System.out.print("Apellido: ");
                    String cliLastName = sn.nextLine();
                    System.out.print("ID: ");
                    String cliId = sn.nextLine();
                    System.out.print("ID Usuario: ");
                    String cliUser = sn.nextLine();
                    System.out.print("Número de cuenta: ");
                    String cliAccNum = sn.nextLine();

                    System.out.println("Tipo de cuenta: 1. Ahorro ($1,000 min) | 2. Inversión ($25,000 min)");
                    int type = sn.nextInt();
                    Account newAcc = null;

                    if (type == 1) {
                        System.out.print("Ingrese el porcentaje de ahorro anual: ");
                        double p = sn.nextDouble();
                        newAcc = new SavingAccount(cliAccNum, p);
                        System.out.println("Ingrese el saldo inicial: "
                                + "\n--- Recuerde que el saldo inicial minimo es 1000 ---");
                        double initialDeposit = sn.nextDouble();
                        if (initialDeposit < 1000) {
                            System.out.println("Error: Monto insuficiente para cuenta de ahorro.");
                            break;
                        }
                        if (!newAcc.deposit(initialDeposit)) {
                            System.out.println("Error: No se pudo crear la cuenta de ahorro.");
                            break;
                        }
                    } else {
                        System.out.print("Monto inicial de inversión: ");
```

```java
                        double initial = sn.nextDouble();
                        if (initial < 25000) {
                            System.out.println("Error: Monto insuficiente para cuenta de inversión.");
                            break;
                        }
                        newAcc = new InvestmentAccount(cliAccNum, initial);
                    }

                    Client newCli = new Client(cliName, cliLastName, cliId, cliUser, newAcc);
                    myBank.addClient(newCli);
                    break;

                case 3:
                    System.out.println("\n--- Registro de Inversionista y Oferta ---");
                    System.out.print("Ingrese el nombre del inversionista: ");
                    String investorName = sn.nextLine();
                    System.out.print("Ingrese el ID del inversionista: ");
                    String investorId = sn.nextLine();
                    System.out.print("Ingrese el nivel de confianza (0-100): ");
                    double investmentTrust = sn.nextDouble();
                    System.out.print("Ingrese el retorno esperado (%): ");
                    double returnRate = sn.nextDouble();
                    sn.nextLine();

                    System.out.print("Ingrese nivel de riesgo de la inversion: ");
                    System.out.println("1. Bajo (" + Investment.LOW_RISK + ")");
                    System.out.println("2. Medio (" + Investment.MEDIUM_RISK + ")");
                    System.out.println("3. Alto (" + Investment.HIGH_RISK + ")");
                    int riskOption = sn.nextInt();
                    String riskLevel = Investment.getRiskDescription(riskOption);

                    if (!Investment.isValidRiskLevel(riskLevel, returnRate)) {
                        // System.out.println("Alerta: El riesgo seleccionado no es adecuado para el retorno esperado.");
                        System.out.println("Inversion rechazada. Por favor, ajuste el nivel de riesgo o el retorno esperado.");
                        break;
                    }

                    System.out.print("Ingrese monto minimo de la inversion: ");
                    double amountMinimum = sn.nextDouble();
                    System.out.print("Ingrese duracion en meses: ");
                    int durationMonths = sn.nextInt();
                    sn.nextLine();

                    Investor investor = new Investor(investorId, investorName, investmentTrust, returnRate);
                    Investment investment = new Investment(riskLevel, amountMinimum, durationMonths, investor);
                    myBank.addInvestmentSecure(investment);
```

```
                    break;

                case 4:
                    System.out.print("\n--- Realizar depósito ---");
                    System.out.print("Ingrese su ID de usuario: ");
                    String depUser = sn.nextLine();
                    Client clientDep = myBank.findClientByUserId(depUser);
                    if (clientDep != null) {
                        System.out.print("Confirme su número de cuenta: ");
                        String depAcc = sn.nextLine();
                        System.out.print("Monto a depositar: ");
                        double depAmount = sn.nextDouble();
                        clientDep.depositMoney(depAcc, depUser, depAmount);
                    } else {
                        System.out.println("Usuario no encontrado.");
                    }
                    break;

                case 5:
                    System.out.print("\n--- Realizar retiro ---");
                    System.out.print("Ingrese su ID de usuario: ");
                    String retUser = sn.nextLine();
                    Client clientRet = myBank.findClientByUserId(retUser);
                    if (clientRet != null) {
                        System.out.print("Digite su ID Física: ");
                        String physicalId = sn.nextLine();
                        System.out.print("Confirme su número de cuenta: ");
                        String retAcc = sn.nextLine();
                        System.out.print("Monto a retirar: ");
                        double retAmount = sn.nextDouble();
                        // Llamamos al método que ajustamos en la clase Client
                        clientRet.retireMoney(retAcc, retUser, physicalId, retAmount);
                    } else {
                        System.out.println("Usuario no encontrado.");
                    }
                    break;

                case 6:
                    System.out.println("Catalogo de inversiones");
                    myBank.showAllInvestments();
                    break;

                case 7:
                    System.out.print("\n--- Consultar días de vacaciones ---");
                    System.out.print("Ingrese ID de usuario del empleado: ");
                    String employUser = sn.nextLine();
                    Employed freeEmp = null;
                    for (Employed emply : myBank.getEmployeds()) {
                        if (emply.getUserId().equals(employUser)) {
                            freeEmp = emply;
```

```java
                            break;
                        }
                    }
                    if (freeEmp != null) {
                        System.out.println(
                                "Días de vacaciones para " + freeEmp.getName() + ": " + freeEmp.daysOfVacation());
                    } else {
                        System.out.println("Empleado no encontrado.");
                    }
                    break;

                case 8:
                    System.out.println("\n--- Aplicar Intereses Mensuales ---");
                    System.out.println("Aplicando intereses mensuales a cuentas de ahorro...");
                    double totalInteresAplicado = 0.0;
                    for (Client c : myBank.getClients()) {
                        if (c.getAccount() instanceof SavingAccount) {
                            SavingAccount savingAccount = (SavingAccount) c.getAccount();
                            double interesAplicado = savingAccount.applyMonthlyInterest();
                            totalInteresAplicado += interesAplicado;
                            System.out.printf(
                                    "Cuenta %s: interés aplicado = $%.2f | nuevo saldo = $%.2f%n",
                                    savingAccount.getAccountNumber(),
                                    interesAplicado,
                                    savingAccount.getBalance());
                        }
                    }
                    System.out.printf("Intereses aplicados con éxito. Total aplicado: $%.2f%n", totalInteresAplicado);
                    break;

                case 9:
                    System.out.println("\n--- Salir ---");
                    exit = true;
                    System.out.println("Cerrando sesión...");
                    break;

                default:
                    System.out.println("\n=== Opción no válida. ===");
            }
        }
        sn.close();
    }
}
```